# Assignment No. 12

**Title:** JAVA Exception Handling - Write a program that calculates result of division of 2 numbers. Implement Exception Handling to handle Divide by Zero Exception and display an appropriate message.

**Objective:** To implement exception handling in Java programs for safe division operation and graceful handling of divide-by-zero runtime errors.

**Theory:** An exception in Java is an event that interrupts the normal flow of program execution. For arithmetic operations, dividing by zero throws an `ArithmeticException`. Using `try`, `catch`, and `finally` blocks helps avoid abrupt program termination and improves robustness.

- `try` : contains risky code
- `catch` : handles the raised exception
- `finally` : executes whether exception occurs or not

**Code:**

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter first number: ");
            int num1 = sc.nextInt();

            System.out.print("Enter second number: ")
            int num2 = sc.nextInt();

            int result = num1 / num2;
            System.out.println("Result = " + result);
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: Can
        } catch (Exception e) {
```

```
            System.out.println("Invalid input. Please
        } finally {
            sc.close();
            System.out.println("Program execution com
        }
    }
}
```

**Output:**

```
Enter first number: 25
Enter second number: 0
Exception caught: Cannot divide by zero.
Program execution completed.
```

**Conclusion:** In this way, Java exception handling was implemented to detect and handle divide-by-zero and invalid input conditions, making the program reliable and user-friendly.

---

# Assignment No. 13

**Title:** JAVA Inheritance - Build a class hierarchy for Bank Account (Saving Account, Current Account) with Base Class `BankAccount`. Override `calculateInterest()` method based on specific account type.

**Objective:** To implement method overriding using inheritance in Java and demonstrate runtime polymorphism with bank account types.

**Theory:** Method overriding allows a subclass to provide its own implementation of a method already defined in the parent class. It is used to achieve runtime polymorphism. In this assignment, `SavingAccount` and `CurrentAccount` override `calculateInterest()` of base class `BankAccount` with different interest rates.

**Code:**

```
class BankAccount {
    String accountHolder;
    double balance;

    BankAccount(String accountHolder, double balance)
        this.accountHolder = accountHolder;
        this.balance = balance;
    }

    double calculateInterest() {
        return 0.0;
    }

    void displayDetails() {
        System.out.println("Account Holder: " + accou
        System.out.println("Balance: Rs " + balance);
        System.out.println("Interest: Rs " + calculat
    }
}

class SavingAccount extends BankAccount {
    SavingAccount(String accountHolder, double balanc
        super(accountHolder, balance);
    }
```

```java
    @Override
    double calculateInterest() {
        return balance * 0.04; // 4% annual interest
    }
}

class CurrentAccount extends BankAccount {
    CurrentAccount(String accountHolder, double balan
        super(accountHolder, balance);
    }

    @Override
    double calculateInterest() {
        return balance * 0.01; // 1% annual interest
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount saving = new SavingAccount("Alice
        BankAccount current = new CurrentAccount("Bob

        System.out.println("--- Saving Account ---");
        saving.displayDetails();

        System.out.println("\n--- Current Account ---
        current.displayDetails();
    }
}
```

**Output:**

```
--- Saving Account ---
Account Holder: Alice
Balance: Rs 50000.0
Interest: Rs 2000.0

--- Current Account ---
Account Holder: Bob
Balance: Rs 50000.0
Interest: Rs 500.0
```

**Conclusion:** In this way, method overriding was successfully implemented in Java inheritance. Different account types used their own `calculateInterest()` behavior while sharing a common base class structure.

---

# Assignment No. 14

**Title:** JAVA Method Overloading - Design and develop a Java program for Library Catalog system. Create overloaded methods for adding and searching books using `addBook()` and `searchBook()`.

**Objective:** To understand and implement method overloading in Java by creating multiple `addBook()` and `searchBook()` methods with different parameter lists.

**Theory:** Method overloading allows more than one method with the same name in a class, provided their parameter lists differ by number, type, or order. It improves readability and flexibility by grouping similar operations under one method name.

**Code:**

```java
import java.util.ArrayList;

class Book {
    int id;
    String title;
    String author;

    Book(int id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
    }
}

class LibraryCatalog {
    private ArrayList<Book> books = new ArrayList<>()
    private int nextId = 1;

    // Overloaded addBook() - auto-generated ID
    void addBook(String title, String author) {
        books.add(new Book(nextId++, title, author));
    }

    // Overloaded addBook() - user-provided ID
    void addBook(int id, String title, String author)
        books.add(new Book(id, title, author));
        if (id >= nextId) {
            nextId = id + 1;
```

```
        }
    }

    // Overloaded searchBook() by ID
    void searchBook(int id) {
        for (Book book : books) {
            if (book.id == id) {
                System.out.println("Found by ID -> "
                return;
            }
        }
        System.out.println("Book not found for ID: "
    }

    // Overloaded searchBook() by title
    void searchBook(String title) {
        for (Book book : books) {
            if (book.title.equalsIgnoreCase(title)) {
                System.out.println("Found by Title ->
                return;
            }
        }
        System.out.println("Book not found for title:
    }
}

public class Main {
    public static void main(String[] args) {
        LibraryCatalog catalog = new LibraryCatalog()

        catalog.addBook("Clean Code", "Robert C. Mart
        catalog.addBook(10, "Java: The Complete Refer
        catalog.addBook("Effective Java", "Joshua Blo

        catalog.searchBook(10);
        catalog.searchBook("Effective Java");
        catalog.searchBook("Data Structures");
    }
}
```

**Output:**

```
Found by ID -> 10 | Java: The Complete Reference | Herbe
Found by Title -> 11 | Effective Java | Joshua Bloch
Book not found for title: Data Structures
```

**Conclusion:** In this way, method overloading was implemented using overloaded `addBook()` and `searchBook()` methods. The library

catalog became more readable, flexible, and easier to use.

# Assignment No. 16

**Title:** Write a small Java application to develop Banking Application in which user deposits the amount Rs 1000.00 and then withdraws Rs 400.00 and Rs 300.00. Develop a GUI using Swing.

**Objective:**

1. To develop a simple banking application using Java Swing.
2. To implement deposit and withdrawal operations with balance updates in GUI.

**Theory:** Swing is Java's standard GUI toolkit used to create windows, labels, buttons, and event-driven programs. In this assignment, `JFrame` is used as the main window and `ActionListener` handles button-click events. Balance is maintained as a variable and updated after each transaction.

**Code:**

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class BankingAppGUI extends JFrame implements
    private double balance = 0.0;

    private final JLabel balanceLabel;
    private final JTextArea logArea;
    private final JButton depositButton;
    private final JButton withdraw400Button;
    private final JButton withdraw300Button;

    public BankingAppGUI() {
        setTitle("Banking Application");
        setSize(500, 350);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE
        setLocationRelativeTo(null);

        balanceLabel = new JLabel("Current Balance: R
        balanceLabel.setFont(new Font("Arial", Font.B

        depositButton = new JButton("Deposit Rs 1000"
        withdraw400Button = new JButton("Withdraw Rs
        withdraw300Button = new JButton("Withdraw Rs
```

```
        depositButton.addActionListener(this);
        withdraw400Button.addActionListener(this);
        withdraw300Button.addActionListener(this);

        JPanel buttonPanel = new JPanel();
        buttonPanel.add(depositButton);
        buttonPanel.add(withdraw400Button);
        buttonPanel.add(withdraw300Button);

        logArea = new JTextArea(10, 40);
        logArea.setEditable(false);
        JScrollPane scrollPane = new JScrollPane(logA

        add(balanceLabel, BorderLayout.NORTH);
        add(buttonPanel, BorderLayout.CENTER);
        add(scrollPane, BorderLayout.SOUTH);
    }

    private void updateBalanceLabel() {
        balanceLabel.setText(String.format("Current B
    }

    private void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            logArea.append(String.format("Withdrawn:
            updateBalanceLabel();
        } else {
            logArea.append(String.format("Insufficien
        }
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == depositButton) {
            balance += 1000.0;
            logArea.append("Deposited: Rs 1000.00\n")
            updateBalanceLabel();
        } else if (e.getSource() == withdraw400Button
            withdraw(400.0);
        } else if (e.getSource() == withdraw300Button
            withdraw(300.0);
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            BankingAppGUI app = new BankingAppGUI();
            app.setVisible(true);
        });
```

```
        }
    }
```

**Output:**

```
GUI window opens with buttons:
1) Deposit Rs 1000
2) Withdraw Rs 400
3) Withdraw Rs 300

Transaction Log:
Deposited: Rs 1000.00
Withdrawn: Rs 400.00
Withdrawn: Rs 300.00

Final Balance shown on screen: Rs 300.00
```

**Conclusion:** In this way, a simple Swing-based banking application was implemented with event handling for deposit and withdrawal, and dynamic balance updates.

---

# Assignment No. 17

**Title:** Implement insert and display operations using JDBC Connectivity.

**Objective:**

1. To connect Java application with MySQL database using JDBC.
2. To perform insert and display operations on database records.

**Theory:** JDBC (Java Database Connectivity) is an API used to connect Java applications with relational databases. `Connection` establishes database connectivity, `PreparedStatement` performs parameterized insert queries, and `ResultSet` is used to read selected records.

**Code:**

```
import java.sql.*;

public class JDBCInsertDisplay {
    private static final String URL = "jdbc:mysql://l
    private static final String USER = "root";
    private static final String PASSWORD = "root";

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnec
             Statement st = con.createStatement()) {

            st.executeUpdate("CREATE TABLE IF NOT EXI
                    + "id INT PRIMARY KEY AUTO_INCREM
                    + "name VARCHAR(50),"
                    + "branch VARCHAR(30))");

            insertStudent(con, "Aditi", "CSE");
            insertStudent(con, "Rahul", "IT");

            System.out.println("\n--- Student Records
            displayStudents(con);

        } catch (SQLException e) {
            System.out.println("Database error: " + e
        }
    }
```

```
    private static void insertStudent(Connection con,
        String sql = "INSERT INTO students(name, bran
        try (PreparedStatement ps = con.prepareStatem
            ps.setString(1, name);
            ps.setString(2, branch);
            ps.executeUpdate();
            System.out.println("Record inserted: " +
        }
    }

    private static void displayStudents(Connection co
        String sql = "SELECT id, name, branch FROM st
        try (PreparedStatement ps = con.prepareStatem
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                System.out.println(rs.getInt("id") +
                        + rs.getString("name") + " |
                        + rs.getString("branch"));
            }
        }
    }
}
```

**Output:**

```
Record inserted: Aditi, CSE
Record inserted: Rahul, IT

--- Student Records ---
1 | Aditi | CSE
2 | Rahul | IT
```

**Conclusion:** In this way, JDBC connectivity was used successfully to insert and display database records in a Java application.

---

# Assignment No. 18

**Title:** Implement CRUD operations using JDBC Connectivity.

**Objective:**

1. To perform Create, Read, Update, and Delete operations using JDBC.
2. To understand structured database interaction in Java applications.

**Theory:** CRUD operations are core data management operations in database systems. JDBC provides `PreparedStatement` for secure query execution and `ResultSet` for reading data. Using separate methods for each CRUD operation improves modularity and maintainability.

**Code:**

```
import java.sql.*;

public class JDBCCrudDemo {
    private static final String URL = "jdbc:mysql://l
    private static final String USER = "root";
    private static final String PASSWORD = "root";

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnec
             Statement st = con.createStatement()) {

            st.executeUpdate("CREATE TABLE IF NOT EXI
                    + "id INT PRIMARY KEY,"
                    + "name VARCHAR(50),"
                    + "price DOUBLE)");

            createRecord(con, 101, "Keyboard", 899.0)
            createRecord(con, 102, "Mouse", 499.0);

            System.out.println("\nAfter Insert:");
            readRecords(con);

            updateRecord(con, 102, 549.0);
            System.out.println("\nAfter Update:");
            readRecords(con);

            deleteRecord(con, 101);
```

```
            System.out.println("\nAfter Delete:");
            readRecords(con);

        } catch (SQLException e) {
            System.out.println("Database error: " + e
        }
    }

    private static void createRecord(Connection con,
        String sql = "INSERT INTO products(id, name,
        try (PreparedStatement ps = con.prepareStatem
            ps.setInt(1, id);
            ps.setString(2, name);
            ps.setDouble(3, price);
            ps.executeUpdate();
            System.out.println("Inserted -> " + id +
        }
    }

    private static void readRecords(Connection con) t
        String sql = "SELECT id, name, price FROM pro
        try (PreparedStatement ps = con.prepareStatem
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                System.out.println(rs.getInt("id") +
                        + rs.getString("name") + " |
                        + rs.getDouble("price"));
            }
        }
    }

    private static void updateRecord(Connection con,
        String sql = "UPDATE products SET price = ? W
        try (PreparedStatement ps = con.prepareStatem
            ps.setDouble(1, newPrice);
            ps.setInt(2, id);
            ps.executeUpdate();
            System.out.println("Updated product ID "
        }
    }

    private static void deleteRecord(Connection con,
        String sql = "DELETE FROM products WHERE id =
        try (PreparedStatement ps = con.prepareStatem
            ps.setInt(1, id);
            ps.executeUpdate();
            System.out.println("Deleted product ID "
        }
    }
}
```

**Output:**

```
Inserted -> 101 | Keyboard | 899.0
Inserted -> 102 | Mouse | 499.0

After Insert:
101 | Keyboard | 899.0
102 | Mouse | 499.0

Updated product ID 102 with new price 549.0

After Update:
101 | Keyboard | 899.0
102 | Mouse | 549.0

Deleted product ID 101

After Delete:
102 | Mouse | 549.0
```

**Conclusion:** In this way, all CRUD operations were implemented successfully using JDBC connectivity with proper query execution and result handling.

---

# Assignment No. 19

**Title:** Design a user input form using Swing components. Implement an `ActionListener` to display user input on button click.

**Objective:**

1. To design a GUI input form using Swing components.
2. To implement event handling using `ActionListener` and display entered data.

**Theory:** Form-based GUI applications are built with components such as `JLabel`, `JTextField`, `JButton`, and `JTextArea`. When the submit button is clicked, `ActionListener` captures the text field values and displays them in output area.

**Code:**

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class UserInputForm extends JFrame implements
    private final JTextField nameField;
    private final JTextField emailField;
    private final JTextField mobileField;
    private final JTextArea outputArea;
    private final JButton displayButton;

    public UserInputForm() {
        setTitle("User Input Form");
        setSize(450, 350);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE
        setLocationRelativeTo(null);
        setLayout(new GridLayout(5, 2, 8, 8));

        add(new JLabel("Name:"));
        nameField = new JTextField();
        add(nameField);

        add(new JLabel("Email:"));
        emailField = new JTextField();
        add(emailField);

        add(new JLabel("Mobile:"));
        mobileField = new JTextField();
```

```
        add(mobileField);

        displayButton = new JButton("Display Input");
        displayButton.addActionListener(this);
        add(displayButton);

        outputArea = new JTextArea();
        outputArea.setEditable(false);
        add(new JScrollPane(outputArea));
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        String name = nameField.getText();
        String email = emailField.getText();
        String mobile = mobileField.getText();

        outputArea.setText("Name: " + name + "\n"
                + "Email: " + email + "\n"
                + "Mobile: " + mobile);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            UserInputForm form = new UserInputForm();
            form.setVisible(true);
        });
    }
}
```

**Output:**

```
GUI Form Input:
Name = Arnav
Email = arnav@example.com
Mobile = 9876543210

Displayed in output area after button click:
Name: Arnav
Email: arnav@example.com
Mobile: 9876543210
```

**Conclusion:** In this way, a Swing user input form was created and user-entered data was successfully displayed using `ActionListener` on button click.

---

# Assignment No. 20

**Title:** Develop a basic calculator application using Swing.

**Objective:**

1. To design a calculator GUI using Swing components.
2. To implement arithmetic operations through button event handling.

**Theory:** A calculator GUI uses a display component and a grid of buttons for digits and operators. Event handling captures user inputs and performs arithmetic calculations. The program stores the first operand, operator, and second operand to evaluate expressions.

**Code:**

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class SimpleCalculator extends JFrame implemen
    private final JTextField display;
    private double firstNumber = 0;
    private String operator = "";
    private boolean startNewNumber = true;

    public SimpleCalculator() {
        setTitle("Basic Calculator");
        setSize(320, 420);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        display = new JTextField("0");
        display.setFont(new Font("Arial", Font.BOLD,
        display.setHorizontalAlignment(JTextField.RIG
        display.setEditable(false);
        add(display, BorderLayout.NORTH);

        JPanel buttonPanel = new JPanel(new GridLayou
        String[] buttons = {
                "7", "8", "9", "/",
                "4", "5", "6", "*",
                "1", "2", "3", "-",
                "C", "0", "=", "+"
```

```
        };

        for (String text : buttons) {
            JButton button = new JButton(text);
            button.setFont(new Font("Arial", Font.BOL
            button.addActionListener(this);
            buttonPanel.add(button);
        }

        add(buttonPanel, BorderLayout.CENTER);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        String command = e.getActionCommand();

        if (command.charAt(0) >= '0' && command.charA
            if (startNewNumber || display.getText().e
                display.setText(command);
                startNewNumber = false;
            } else {
                display.setText(display.getText() + c
            }
        } else if (command.equals("C")) {
            firstNumber = 0;
            operator = "";
            display.setText("0");
            startNewNumber = true;
        } else if (command.equals("=")) {
            double secondNumber = Double.parseDouble(
            double result = 0;

            switch (operator) {
                case "+": result = firstNumber + seco
                case "-": result = firstNumber - seco
                case "*": result = firstNumber * seco
                case "/":
                    if (secondNumber == 0) {
                        JOptionPane.showMessageDialog
                        display.setText("0");
                        startNewNumber = true;
                        return;
                    }
                    result = firstNumber / secondNumb
                    break;
                default:
                    result = secondNumber;
            }

            display.setText(String.valueOf(result));
            startNewNumber = true;
        } else {
```

```
            firstNumber = Double.parseDouble(display.
            operator = command;
            startNewNumber = true;
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            SimpleCalculator calc = new SimpleCalcula
            calc.setVisible(true);
        });
    }
}
```

**Output:**

```
Sample calculations performed in GUI:
12 + 8 = 20.0
9 * 7 = 63.0
25 / 5 = 5.0
```

**Conclusion:** In this way, a basic calculator was successfully developed using Swing, implementing arithmetic operations through event-driven programming.